Investigating Sparse Allreduce Performance in MPI

Chuao Dong

Abstract

This project implements a sparse Allreduce in C++ and MPI that sends only the nonzero entries of each vector, stored as sorted (index, value) pairs, instead of the full dense vector. Following the Kylix design [1], the processes are organized into a one- or two-layer network. In each layer, the processes in a group split the index range among themselves, send every pair to the process that owns its index with `MPI_Alltoallv`, and add up the pairs they receive with a heap-based k-way merge, which keeps the output sorted without a separate sort. A final `MPI_Allgatherv` gives every process the complete sum. We compare this implementation with `MPI_Allreduce` on the equivalent dense vectors on a single 128-core node, varying vector length, sparsity, number of processes, and network layout. Each experiment is run with two kinds of input: one in which all processes have nonzeros at the same indices, and one in which each process chooses its own. When the nonzeros line up, the sparse version wins clearly for large, sparse vectors: with 64 processes, a vector length of 5 million per process, and 99% sparsity, it is $10.4\times$ faster than `MPI_Allreduce` with one layer and $13.2\times$ faster with two layers. When each process chooses its own indices, the sum fills in and becomes much denser than any input, and the same configuration is only a tie ($1.15\times$). The sparse version then needs sparser inputs to win, and even sparser as more processes are added.

1 Introduction

Allreduce serves as a fundamental collective operation in parallel and distributed computing. In an Allreduce, data from each process in a distributed system, typically represented as a vector v_i for process i, is aggregated (say via a sum) and the result is redistributed to all processes. It plays a pivotal role in parallel algorithms where individual processes compute partial solutions that must be combined, and efficient implementations are imperative for large-scale parallelism.

In numerous applications, these vectors are sparse. Here, each process holds a sparse vector V_i, represented as an array of (index, value) pairs in which only nonzero values are stored. The Allreduce produces a single, potentially sparse vector V, replicated on all processes, in which each index j holds the sum of $V_i[j]$ over all processes with $V_i[j] \neq 0$. If no process has a nonzero value at index j, $V[j]$ is 0 and is not stored.

This project implements a variant of Kylix, a nested, heterogeneous-degree butterfly network proposed by Zhao and Canny [1], and compares it with `MPI_Allreduce` applied to the equivalent dense vectors. Sending only nonzeros reduces communication volume, but sparse Allreduce has two costs that dense Allreduce does not: each pair carries its index as well as its value, and the sum of sparse vectors can be much denser than any input. When processes (MPI ranks) hold different nonzero patterns, this *fill-in* can make the result dense enough that the sparse representation costs more than the dense one.

The study therefore runs every experiment under two patterns that bracket real workloads: a *shared* pattern, in which all ranks hold the same indices of nonzero values and the result is no denser than each input, and *independent* patterns, in which each rank independently draws its own indices of nonzero values. The project has two objectives:

Final project report for CS 267 Spring 2024.

- to measure how vector length, sparsity, process count, and network layout affect the performance of Sparse Allreduce under both patterns;
- to find the *crossover* sparsity, at which sparse and dense take equal time, as a function of process count.

2 Implementation

2.1 Sparse Vector

The sparse vector is represented by a structure comprising three members: an integer `length` for the vector length, an integer `nnz` for the number of nonzero elements, and an array `data` of (index, value) pairs sorted by index. Each pair is a structure with `idx` (index) and `val` (value) members, transmitted using a custom MPI datatype.

2.2 Layer

The Allreduce network is structured into l layers. Processes within layer $i \in \{1, \ldots, l\}$ are grouped into sets of size d_i, the degree of layer i, and the product of the degrees equals the number of processes. Each layer is defined by a structure containing `in_degree` (the degree of the preceding layer), `out_degree` (its own degree), and `depth` (its position in the network). This project implements single-layer and two-layer networks.

2.3 Sparse Allreduce

The Sparse Allreduce operation comprises three sequential steps: partition, Scatter-Reduce, and Allgather.

2.3.1 Partition

On layer i, processes are grouped into sets of size d_i using `MPI_Comm_split`. If $i > 1$, only processes holding the same range of indices are grouped together.

2.3.2 Scatter-Reduce

During this phase, communication is confined within process groups.

Scatter: The index range owned by the group is divided into equal-sized parts, one per group member. Each process partitions its pairs by these ranges, exchanges pair counts with `MPI_Alltoall`, and sends each part to its owner with `MPI_Alltoallv`.

Reduce: Each process receives one sorted segment from every group member and merges them with a k-way merge on a priority queue, where k is the group size. The queue is initialized with the first pair of each segment; pairs with equal indices are summed, sums of zero are dropped, and the next pair from the same segment is inserted until all segments are consumed. This exploits the sorted order of each segment to produce sorted output without a separate sorting step, and the queue holds at most one entry per nonempty segment. Sums are accumulated in 64-bit integers and rejected if they exceed the stored integer range.

2.3.3 Allgather

Each process now holds the fully reduced pairs for one index range. Processes exchange segment lengths with `MPI_Allgather` and collect all segments with `MPI_Allgatherv`. With one layer, process order matches index order, so the gathered array is already sorted. With two layers, each process still owns one disjoint index range, but process order no longer matches index order. The gathered segments are therefore reordered by their first index and concatenated, which takes linear time.

Figure 1 illustrates a 3×2 network for 6 processes, where each process communicates with three neighbors (including itself) on layer 1 and two neighbors (including itself) on layer 2.

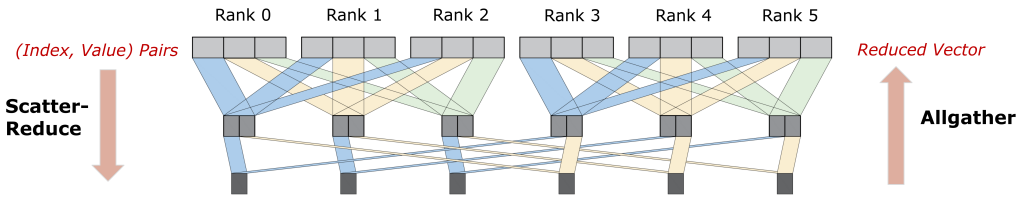


Figure 1: 3×2 Butterfly Network for Sparse Allreduce.

2.4 Sparse Vector Generator

Each index is included independently with probability equal to one minus the requested sparsity, so nonzero positions are uniform and the realized nonzero count is approximate. All nonzero values are 1. With a *shared* pattern, every rank seeds its generator identically and holds the same indices. With *independent* patterns, each rank mixes its rank into the seed and draws its own indices. If each rank's nonzero fraction is q, an index of the sum is zero only when all p ranks miss it, so the expected nonzero fraction of the result is

$$\rho_{\text{out}} = 1 - (1 - q)^p. \tag{1}$$

At 99% sparsity and 64 ranks, for example, the result is about 47% nonzero.

3 Experimental Methodology

The experiments run on a single 128-core CPU node, with MPI processes bound to CPU cores. The process-count experiments use up to 128 ranks and a fixed vector length per rank; they measure single-node behavior, not multi-node scaling. Sparsity levels are 90%, 95%, 98%, 99%, 99.5%, and 99.9%. Each experiment runs once with shared and once with independent patterns, as separate batch jobs.

Each configuration runs three untimed warmup iterations followed by five measured iterations in the same MPI session, each starting from the original input. Sparse and dense operations are timed separately with `MPI_Wtime`, each after a barrier. For each operation and iteration, the maximum elapsed time across ranks is collected; the reported time is the median of the five maxima. The sparse timer includes communicator creation and release, Scatter-Reduce, Allgather, and final result assembly. Input generation, dense conversion, barriers, timing aggregation, and correctness checks are excluded. Sparse results are checked against dense `MPI_Allreduce` on every rank in every iteration, including warmups; all 1,032 recorded runs pass.

The runtime breakdown uses a separate profiling executable with three warmups and one measured iteration. All component times and the total come from rank zero in that iteration. This gives additive timings for the stacked plots, describing rank zero rather than the slowest rank, and a single sample does not characterize variability.

Dense `MPI_Allreduce` is markedly faster at power-of-two process counts, for example, with a vector length of 1M, 2.9 ms at 8 ranks against 4.8 ms at 6. Speedups at 6, 12, 18, and 24 ranks partly reflect slow dense runs, so power-of-two counts give the cleanest comparisons.

4 Runtime Breakdown Analysis

The profiler divides sparse time into scatter time (count exchange and `MPI_Alltoallv`), reduce time (local merges and final result assembly), Allgather time (count exchange and `MPI_Allgatherv`), and a remainder that combines communicator splitting with work outside the component timers. Figures 2 and 3 show this breakdown for a single-layer network with vector length 1M.

With either pattern, reduce time grows between 2 and 128 processes at every sparsity level, while its share of the total falls. Each rank contributes a fixed-size input, so adding ranks adds input pairs; under a balanced partition, each rank receives a roughly constant number of pairs, but split across more segments, which lengthens the merge.

The patterns differ in the Allgather. With a shared pattern, the gathered result is as sparse as each input, and Allgather takes 14% to 32% of the total at 64 and 128 ranks. With independent patterns, the result fills in according to Equation 1, and the Allgather share rises from 14–29% at 2 ranks to 66–91% at 128. At 64 ranks and 99% sparsity, Allgather takes 6.8 ms of 8.2 ms with independent patterns, against 0.6 ms of 1.8 ms with a shared pattern.

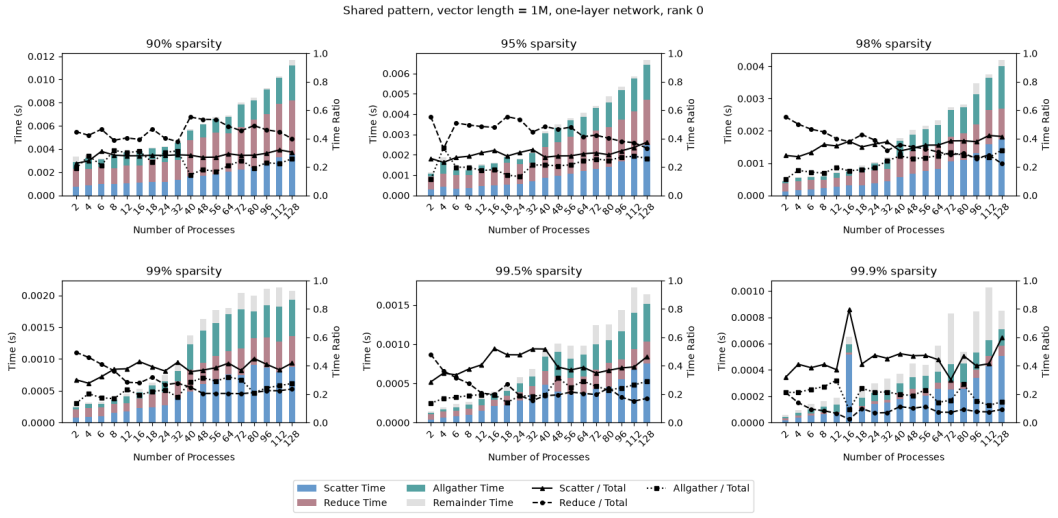


Figure 2: Runtime breakdown with a shared pattern, single-layer network, vector length 1M, rank 0.

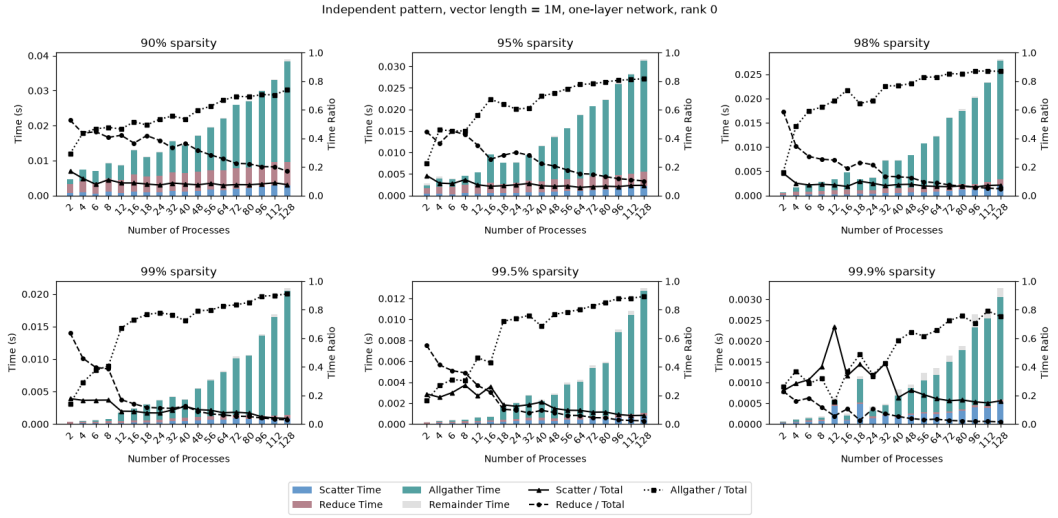


Figure 3: Runtime breakdown with independent patterns, single-layer network, vector length 1M, rank 0.

5 Performance

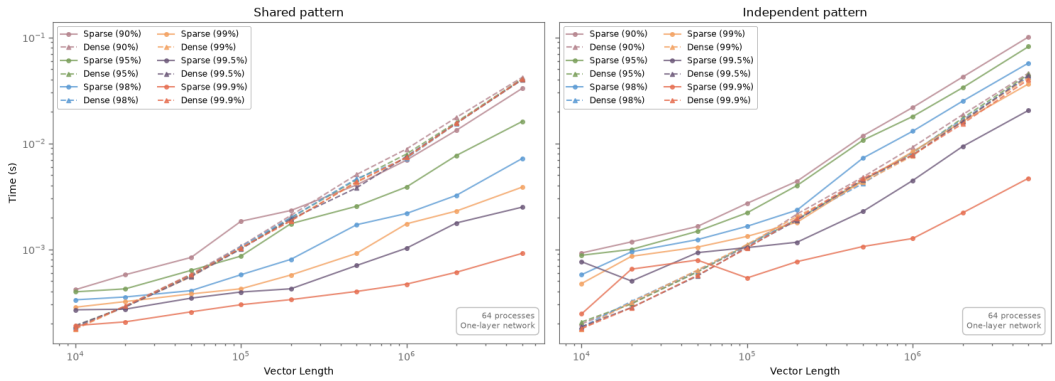
5.1 Sparsity Level and Vector Length

Figure 4 compares Sparse Allreduce with dense `MPI_Allreduce` across vector lengths and sparsity levels with 64 processes, and Table 1 lists the speedups at a vector length of 5M per rank. Dense time is independent of sparsity, since the dense message size depends only on vector length. Sparse time falls as sparsity rises.

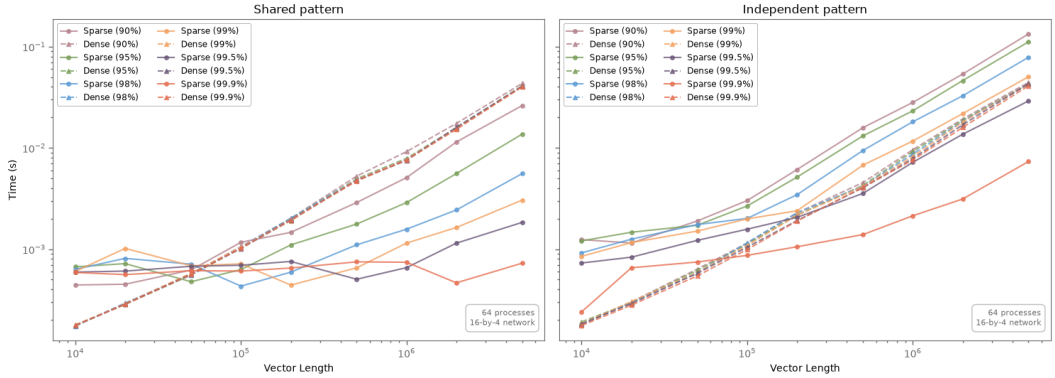
Table 1: Speedup (dense time / sparse time) at 64 ranks and a vector length of 5M per rank.

Pattern	Network	90%	95%	98%	99%	99.5%	99.9%
Shared	One layer	1.27	2.53	5.60	10.40	16.02	44.21
Shared	16×4	1.66	3.00	7.24	13.16	22.15	55.34
Independent	One layer	0.46	0.55	0.74	1.15	2.14	8.54
Independent	16×4	0.33	0.38	0.55	0.85	1.48	5.60

With a shared pattern, sparse Allreduce has fixed costs that favor dense at small lengths, but wins at every sparsity level for sufficiently long vectors: with one layer, from a vector length of 20K at 99.5% and above, 50K at 98% and 99%, 100K at 95%, and 500K at 90%. With independent patterns, it never wins at 98% sparsity or below; at 99% the single-layer network ties dense from a vector length of 2M onward, and at 99.5% and 99.9% it wins from a vector length of 100K.



(a) One-layer network



(b) Two-layer 16×4 network

Figure 4: Sparse and dense Allreduce time across vector lengths and sparsity levels with 64 processes.

5.2 Scalability

Figures 5 and 6 show times and speedups across process counts for vectors of length 1M with a single-layer network. With a shared pattern, sparse Allreduce wins at all 18 tested process counts from 95% sparsity upward and at 14 of 18 at 90%. With independent patterns, it wins at all 18 counts only at 99.5% and 99.9%; at 99% it wins at 16 of 18, at 98% at 6, at 95% at 1, and at 90% at none. Speedups with independent patterns fall as process count grows, because the result fills in further.

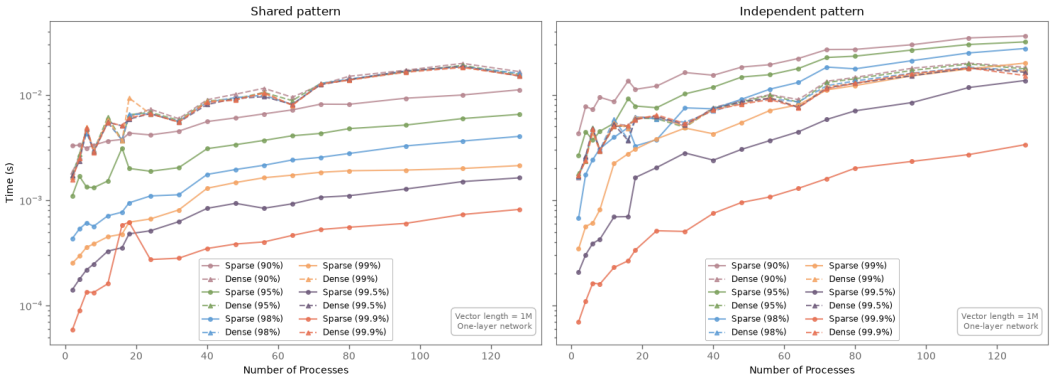


Figure 5: Sparse (single-layer) and dense Allreduce time across process counts and sparsity levels, vector length 1M.

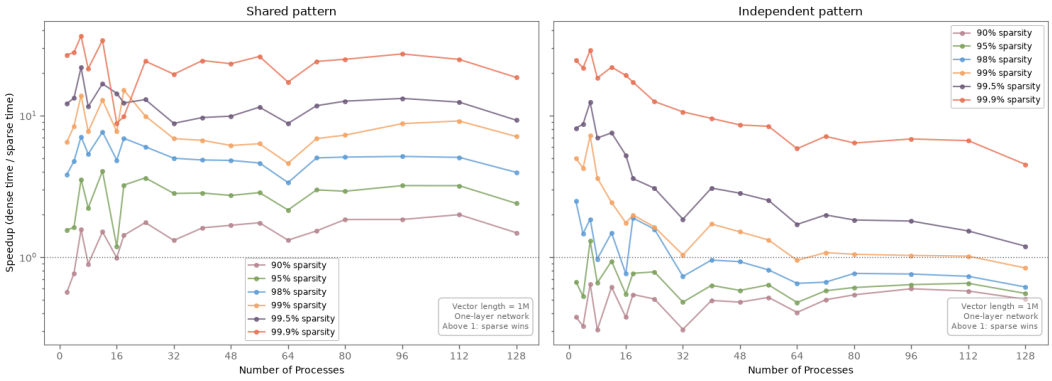


Figure 6: Speedup across process counts and sparsity levels, single-layer network, vector length 1M. Values above 1 mean sparse wins.

5.3 Network Layers

Compared with the single-layer network, the two-layer network adds a stage of communicator setup, communication, and reduction, but each Scatter-Reduce step communicates within smaller groups. Figure 7 compares the two networks across vector lengths with 64 processes, and Figure 8 across process counts with a vector of length 1M; the latter runs both networks in the same job, so both are measured under the same conditions and the comparison is more reliable.

The first runs with independent patterns showed the two-layer network taking 2.3 to 3.2 times as long as one layer. At 64 ranks, the excess over one layer was close to a constant 26–37 ns per pair of the result, which pointed to work proportional to the result size. The cause was the final step: the gathered segments were merged with the priority queue, an $O(n \log p)$ operation over the whole result on every rank, although each segment is a disjoint index range and only their order needs restoring. Replacing the merge with the reorder described in Section 2.3.3 reduced two-layer time at 64 ranks by 39% to 49% across sparsity levels, for example from 20.7 ms to 11.7 ms at 99%, against 8.2 ms for one layer.

After this change, the two-layer network is faster than one layer in 57 of the 96 configurations of the process-count comparison with shared patterns, up from 32, with a median advantage of 19% to 27% at 98% to 99.5% sparsity. With independent patterns, one layer is still faster in 95 of 96 configurations; two layers typically take 30–60% longer. The likely reason is the second exchange round in the two-layer network: after the first exchange round, each process holds partial sums that have already filled in, so the second round still has a large amount of data to send, whereas with a shared pattern it sends very little.

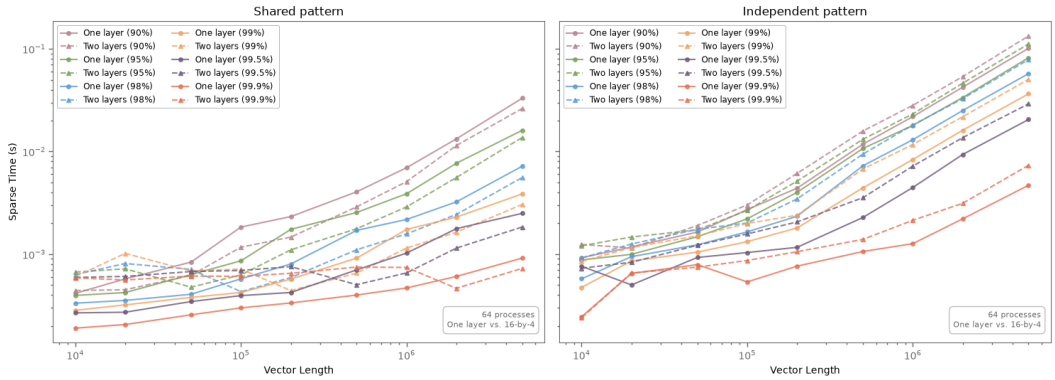


Figure 7: One-layer versus 16×4 sparse time across vector lengths and sparsity levels with 64 processes.

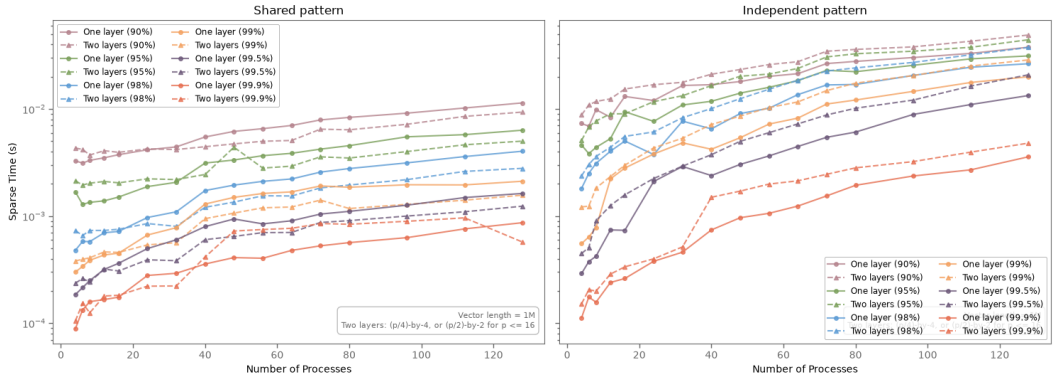


Figure 8: One-layer versus two-layer sparse time across process counts and sparsity levels, vector length 1M. Two-layer networks are $(p/4) \times 4$, or $(p/2) \times 2$ for $p \leq 16$.

6 Fill-in and the Crossover

Figure 9 compares the measured nonzero fraction of the result with Equation 1, using each run's measured mean input density. The prediction matches all 108 independent process-count runs within 0.07 percentage points; with a shared pattern, the result density equals the input density.

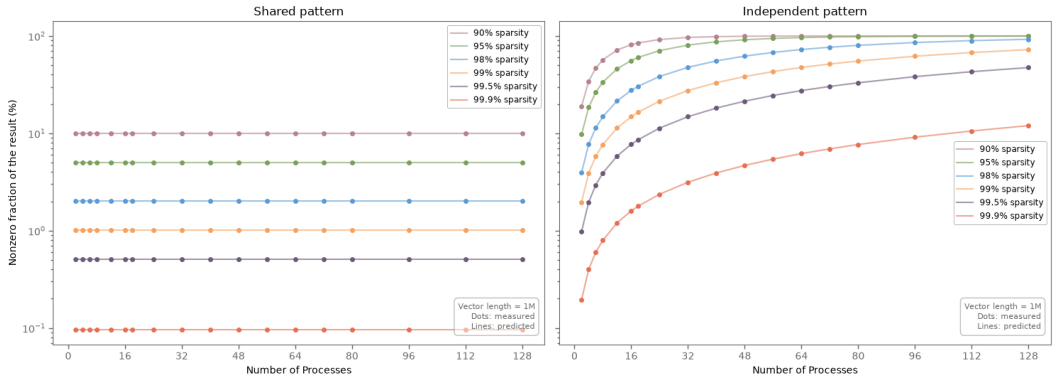


Figure 9: Nonzero fraction of the result across process counts, vector length 1M. Dots are measured; lines are Equation 1.

Only six sparsity levels are tested, so for each process count the crossover is reported as a range: between the last level at which sparse loses and the first level from which it wins. With a shared

pattern, sparse already wins at 90%, the lowest level tested, at 14 of 18 process counts, and the crossover lies between 90% and 95% at the other four. With independent patterns, the crossover rises with process count (Table 2). Near the crossover, speedups are close to 1: at 32 ranks sparse wins at 99% by only $1.04\times$, and at 72 to 112 ranks by $1.02\times$ to $1.08\times$, which are effectively ties.

Table 2: Crossover sparsity range, single-layer network, vector length 1M. $\leq 90\%$ means sparse already wins at 90%, the lowest level tested.

Ranks	2	4	8	16	32	64	128
Shared	90–95%	90–95%	90–95%	90–95%	$\leq 90\%$	$\leq 90\%$	$\leq 90\%$
Independent	95–98%	95–98%	98–99%	98–99%	98–99%	99–99.5%	99–99.5%

With independent patterns, the crossover rises with process count because every added process puts its nonzeros in new places, so the sum gets denser. At 99% sparsity, for example, the sum is about 8% nonzero with 8 processes but about 72% with 128, and the sparse version goes from winning by $3.6\times$ to losing ($0.84\times$). To keep the sum small enough for the sparse version to win, the inputs must be sparser, and the more processes there are, the sparser they must be. Dense `MPI_Allreduce` also gets slower with more processes (about 3 ms at 8 processes, 17 ms at 128), which helps the sparse version a little, but not enough to make up for the fill-in.

7 Summary

Sending only nonzeros pays off when nonzero patterns overlap across ranks. With a shared pattern, sparse Allreduce beats dense `MPI_Allreduce` at every tested sparsity level for vectors of moderate length, reaching $10.4\times$ with one layer and $13.2\times$ with a 16×4 network at 99% sparsity, 64 ranks, and a vector length of 5M per rank. With independent patterns, fill-in erodes this advantage: the result density follows $1 - (1 - q)^p$ almost exactly, the Allgather of the filled-in result dominates the run time, and the sparsity needed to win rises from 95–98% at 2 and 4 ranks to 99–99.5% at 64 and 128 ranks. The crossover rises with process count because fill-in grows faster than dense Allreduce time. Real workloads, in which popular indices recur across ranks, should fall between the two patterns.

On the implementation side, profiling under fill-in exposed a redundant merge in the two-layer network; replacing it with a linear-time reorder cut two-layer time by about 45% and made the two-layer network faster than one layer in most shared-pattern configurations.

The results are limited to one node, uniform index positions, and integer values. Natural extensions are multi-node runs, where communication between nodes is slower than within one node; skewed index distributions between the two patterns studied here; floating-point values, where the summation order affects reproducibility; and switching to dense `MPI_Allreduce` when the predicted result density is too high for the sparse version to win.

References

[1] Huasha Zhao and John Canny. “Kylix: A sparse allreduce for commodity clusters”. In: *2014 43rd International Conference on Parallel Processing*. IEEE. 2014, pp. 273–282.